



DHBW Mannheim

TINF24CS2 · Modul Software Engineering

StudyMate v2

Projektdokumentation

GRUPPE

Selina Ickstadt
Devi Müller
Michelle Schneider
Jennifer Hirt

DOZENT

Jürgen Schultheis

ABGABEDATUM

26.06.2026

Inhaltsverzeichnis

1. Projektziel und Überblick

Kernziele

Implementierte Hauptfunktionen

2. User Stories

Epic 1: Benutzerverwaltung

Epic 2: Lernset-Verwaltung

Epic 3: Karteikarten-Verwaltung

Epic 4: Lernen und Quiz

Epic 5: Entdecken und soziale Funktionen

Epic 6: Onboarding und UX

3. Architekturübersicht

Schichtenarchitektur

Technologie-Stack

4. Projektstruktur

5. Datenbankschema

Tabellen

SQL-Referenzschema (aus `sql_schema.sql`)

6. Backend — Komponenten und API-Referenz

6.1 `app/main.py` — Anwendungsinitialisierung

6.2 `app/core/config.py` — Konfiguration

6.3 `app/core/security.py` — Authentifizierung und Autorisierung

6.4 API-Endpunkte — Vollständige Referenz

7. Frontend — Komponenten und Funktionen

7.1 Hilfsfunktionen

7.2 Navigations-Zustände (view)

7.3 Hauptkomponenten

8. Authentifizierung und Sicherheit

Auth-Mechanismus

Sicherheitsrichtlinien

9. Feature-Dokumentation (chronologisch)

Phase 1 — Grundfunktionen (2026-05-19)

Phase 2 — Erweiterungen (2026-05-26)

Phase 3 — Bugfixes und Verbesserungen (2026-05-26 — 2026-06-01)

Phase 4 — Soziale Funktionen, KI-Anbieter-Wechsel, UI-Feinschliff (2026-06-01 — 2026-06-09)

Phase 5 — Sicherheits-Fix Passwort-Reset (2026-06-23)

10. Konfiguration und Umgebungsvariablen

Backend (.env in backend/)

Frontend (.env.local in frontend/)

11. Lokale Entwicklungsumgebung

11.1 Backend starten

11.2 Frontend starten

12. UI/UX — Design und Animationen

Design-System

Flip-Card-Animation

Responsive Design

13. CI/CD und Deployment

Deployment-Konfiguration

Empfohlener CI/CD-Workflow

Branching-Strategie (Aktuell)

14. Tests und Qualitätssicherung

Aktueller Stand

Durchgeführte manuelle Tests

Bugfix: Fork-Funktion kopiert Karten ohne Position (2026-06-01)

Automatisierte E2E-Tests mit Playwright (ab 2026-06-01)

Empfohlene Backend-Tests (zukünftig, pytest)

15. Troubleshooting und FAQ

Problem: CORS-Fehler beim API-Aufruf

Problem: Supabase Token-Fehler (401)

Problem: Passwort-Reset-Code wird abgelehnt ("Code ungültig oder abgelaufen")

Problem: Flip-Card zeigt Vor- und Rückseite gleichzeitig (Firefox)

Problem: KI-Quiz schlägt fehl

Problem: Passwort-Reset E-Mail kommt nicht an

Problem: Fork-Funktion gibt Fehler zurück

16. ChangeLog

Version 0.3.2 (2026-06-23) — Aktuelle Version

Version 0.3.1 (2026-06-09)

Version 0.3.0 (2026-06-02)

Version 0.2.2 (2026-06-01)

Version 0.2.1 (2026-06-01)

Version 0.2.0 (2026-06-01)

Version 0.1.0 (2026-05-19) — Erstveröffentlichung

17. Offene Aufgaben

18. Anhänge

A) Vollständiges SQL-Referenzschema

B) Pydantic-Schemas (Übersicht)

C) Nützliche curl-Befehle

D) JSON-Import-Format für Karteikarten

1. Projektziel und Überblick

StudyMate v2 ist eine vollständige, modern gestaltete Web-Lernplattform, die sich auf Karteikarten-basiertes Lernen spezialisiert. Das Projekt wurde im Rahmen des Software-Engineering-Moduls (WS 2025/26) als kollaboratives Teamprojekt entwickelt.

Kernziele

- Bereitstellung einer leichtgewichtigen, erweiterbaren Plattform für Karteikarten-basiertes Lernen
- Praxisnahe Anwendung moderner Software-Engineering-Methoden (agile Entwicklung, Git-Workflow, CI/CD)
- Integration externer Dienste (Supabase, Groq KI, Resend E-Mail)

Implementierte Hauptfunktionen

Funktion	Beschreibung	Implementiert am
Authentifizierung	Login, Registrierung, Gastmodus, Passwort-Reset	2026-05-19
Lernset-Verwaltung	Erstellen, Bearbeiten (Titel, Beschreibung), Löschen, Sichtbarkeit umschalten	2026-05-19 / 2026-05-26
Karteikarten-Verwaltung	Karten anlegen, bearbeiten, löschen, JSON-Import, Reihenfolge ändern	2026-05-19 / 2026-06-02
Lernmodus	Flip-Card-Ansicht mit Fortschrittsanzeige und Streak	2026-05-19
Standard-Quiz	Multiple-Choice auf Basis vorhandener Karten	2026-05-19
KI-Quiz	Quiz-Generierung via Groq API (Llama) mit Erklärungen	2026-05-26 / 2026-06-02
Fork-Funktion	Öffentliche und eigene Sets kopieren (inkl. Priv-zu-Priv)	2026-05-26 / 2026-06-02
Favoritensystem	Sets als Favorit markieren (auch als Gast), Migration bei Anmeldung	2026-05-19 / 2026-06-02
Streak-Tracking	Tägliche Lernserie lokal im Browser gespeichert	2026-05-26
Geführte Tour	Interaktive Erstbenutzerschulung (Dashboard, Set-Ansicht, Set-Erstellung)	2026-05-26 / 2026-06-02
Profil-Overlay	Profilansicht mit Statistiken und Bearbeitungsmöglichkeit	2026-05-26
Profilbild in NavBar/Sidebar	Hochgeladenes Profilbild erscheint im Avatar der Navbar und Sidebar-Fußzeile	2026-06-01
Light/Dark-Mode	Umschaltbares Farbschema (gespeichert per localStorage)	2026-05-26

2. User Stories

User Stories beschreiben die Anforderungen aus Benutzerperspektive. Sie wurden als Grundlage für die Entwicklung verwendet und bilden die Basis der Akzeptanzkriterien.

Epic 1: Benutzerverwaltung

US-001 — Registrierung

Als neuer Benutzer möchte ich mich mit Benutzername und Passwort registrieren können, damit ich ein persönliches Konto für meine Lernsets erhalte.

Akzeptanzkriterien: - Benutzername muss mindestens 3 Zeichen lang sein - Benutzername darf nur Buchstaben, Zahlen und `_` enthalten - Passwort muss mindestens 6 Zeichen lang sein - Bei bereits vergebenem Benutzernamen wird eine verständliche Fehlermeldung angezeigt - Nach erfolgreicher Registrierung wird der Benutzer automatisch eingeloggt und zum Dashboard weitergeleitet

Implementiert: 2026-05-19 — Supabase Auth-Integration

Datei: [frontend/src/StudyMate.jsx](#) (Funktion `AuthView`, `handleRegister`)

US-002 — Login

Als registrierter Benutzer möchte ich mich mit Benutzername und Passwort einloggen können, damit ich auf meine gespeicherten Lernsets zugreifen kann.

Akzeptanzkriterien: - Login funktioniert mit korrektem Benutzernamen und Passwort - Bei falschem Login erscheint eine benutzerfreundliche Fehlermeldung (kein technisches Detail) - Sitzung bleibt bis zum manuellen Logout oder Token-Ablauf aktiv

Implementiert: 2026-05-19

Datei: [frontend/src/StudyMate.jsx](#) (Funktion `handleLogin`)

US-003 — Gastmodus

Als Besucher möchte ich die Plattform ohne Registrierung ausprobieren können, damit ich öffentliche Lernsets ansehen kann, bevor ich mich anmelde.

Akzeptanzkriterien: - Ein „Als Gast fortfahren“-Button ist auf der Login-Seite sichtbar - Im Gastmodus sind nur öffentliche Sets sichtbar - Aktionen wie Set-Erstellen leiten den Gast zur Anmeldeseite - Gäste können Sets als Favoriten markieren; diese werden bei der Anmeldung automatisch ins Benutzerkonto übernommen

Implementiert: 2026-05-19

Datei: [frontend/src/StudyMate.jsx](#) (Funktion `handleGuest`)

US-004 — Passwort zurücksetzen

Als Benutzer möchte ich mein vergessenes Passwort zurücksetzen können, damit ich wieder Zugriff auf mein Konto erhalte.

Akzeptanzkriterien: - Über „Passwort vergessen?“ gelangt man zur Reset-Seite - Eingabe des Benutzernamens sendet einen 6-stelligen Code an die hinterlegte Recovery-E-Mail - Die Antwort des Servers gibt keine Information darüber, ob ein Benutzer existiert (Schutz vor Enumeration) - Der Code ist 1 Stunde gültig und wird direkt im Formular eingegeben (kein Klick-Link)

Implementiert: 2026-05-19 ; auf OTP-Code umgestellt: 2026-06-23

Datei: [backend/app/routers/auth.py](#), [frontend/src/components/AuthViews.jsx](#) (Funktion `ForgotPasswordView`)

Hinweis (2026-06-23): Ursprünglich wurde ein klickbarer Magic-Link verwendet. In der Praxis verbrauchten E-Mail-Sicherheitsscanner (z. B. von Gmail/Antivirus-Software) den Einweg-Token bereits beim Öffnen der Mail, bevor der Nutzer selbst klickte — der Link landete dann ohne gültige Session auf der normalen Startseite. Die Umstellung auf einen manuell eingegebenen OTP-Code behebt das, da kein automatisierter Request den Code mehr „verbrauchen“ kann.

US-005 — Profilansicht und -bearbeitung

Als angemeldeter Benutzer möchte ich mein Profil mit Anzeigenamen, Bio und Profilbild verwalten können, damit andere Benutzer mehr über mich erfahren.

Akzeptanzkriterien: - Profilseite zeigt Anzeigenname, Bio, Statistiken (Anzahl Sets, Karten, Streak-Tage) - Anzeigenname und Bio können bearbeitet werden - Profilbild kann hochgeladen oder entfernt werden (lokal gespeichert) - Änderungen werden in Supabase (Anzeigenname) und localStorage (Bio, Bild) gespeichert - Hochgeladenes Profilbild erscheint als Avatar in Navbar und Sidebar-Fußzeile

Implementiert: 2026-05-26 ; Profilbild in NavBar/Sidebar: 2026-06-01

Datei: [frontend/src/StudyMate.jsx](#) (Funktionen `ProfileView` , `ProfileEditView` , `handleSaveProfile`)

Epic 2: Lernset-Verwaltung

US-010 — Lernset erstellen

Als angemeldeter Benutzer möchte ich ein neues Lernset mit Titel, Beschreibung und Sichtbarkeit erstellen können, damit ich meine Lernkarten strukturiert organisieren kann.

Akzeptanzkriterien: - Ein Dialog öffnet sich bei Klick auf „Neues Set“ - Titel ist Pflichtfeld; Beschreibung ist optional - Sichtbarkeit kann zwischen „Privat“ und „Öffentlich“ gewählt werden - Nach Erstellung landet man direkt in der Set-Detailansicht

Implementiert: 2026-05-19

Datei: [frontend/src/StudyMate.jsx](#) (Funktion `handleCreateSet` , `submitCreateSet`)

US-011 — Lernset bearbeiten

Als Eigentümer eines Sets möchte ich Titel und Beschreibung des Sets nachträglich ändern können, damit ich Fehler korrigieren oder das Set umbenennen kann.

Akzeptanzkriterien: - Ein Bearbeitungs-Button ist neben dem Set-Titel in der Detailansicht sichtbar (nur für Eigentümer) - Titel und Beschreibung können geändert und gespeichert werden - Änderungen werden sofort in der Ansicht reflektiert

Implementiert: 2026-05-26 (Commit: `9282ff3 set namen bearbeiten können`)

Datei: [frontend/src/StudyMate.jsx](#) (Funktion `saveEditTitle` , `handleUpdateSetTitle`)

US-012 — Sichtbarkeit umschalten

Als Eigentümer möchte ich ein Set von privat auf öffentlich (und zurück) schalten können, damit ich entscheide, wer meine Sets sehen kann.

Akzeptanzkriterien: - Umschalten nur für Eigentümer möglich - Bestätigungsdialog vor der Änderung - Badge im Set zeigt aktuellen Status (Öffentlich/Privat) - Beim Veröffentlichen fragt ein Dialog, ob der Ersteller (Avatar/Name) auf dem Set sichtbar sein soll (`show_author`); Standard ist sichtbar

Implementiert: 2026-05-19 ; Autoren-Sichtbarkeits-Dialog: 2026-06-02

Datei: [frontend/src/StudyMate.jsx](#) (Funktion `handleToggleSetVisibility`),
[frontend/src/components/DetailView.jsx](#) (`showAuthorDialog`)

US-013 — Lernset löschen

Als Eigentümer möchte ich ein Set endgültig löschen können, wenn ich es nicht mehr benötige.

Akzeptanzkriterien: - Löschen nur für Eigentümer möglich - Bestätigungsdialog vor dem Löschen - Nach dem Löschen wird zur Dashboard-Ansicht zurückgekehrt

Implementiert: 2026-05-19

Datei: [frontend/src/StudyMate.jsx](#) (Funktion `handleDeleteSet`)

US-014 — Öffentliches Set forken

Als angemeldeter Benutzer möchte ich ein öffentliches Set in mein Konto kopieren (forken) können, damit ich es bearbeiten und an meine Bedürfnisse anpassen kann.

Akzeptanzkriterien: - Fork-Button ist bei öffentlichen, fremden Sets sichtbar (nicht für Gäste) - „Set kopieren“-Button ist für eigene Sets sichtbar (öffentlich wie privat) - Ein Dialog erlaubt das Umbenennen des geforkten/kopierten Sets vor dem Speichern - Das geforkte/kopierte Set ist zunächst privat - Alle Karten des Original-Sets werden übernommen

Implementiert: 2026-05-26 (Commits: [98d6cdf](#) , [4a41540](#) , [e46744c](#)); Priv-zu-Priv-Fork: 2026-06-02

Dateien: [backend/app/routers/sets.py](#) (Funktion `fork_set`), [frontend/src/StudyMate.jsx](#) (Funktion `handleForkSet` , `submitForkSet`)

Epic 3: Karteikarten-Verwaltung

US-020 — Karteikarte hinzufügen

Als Eigentümer eines Sets möchte ich neue Karteikarten mit Frage und Antwort hinzufügen können, damit das Set wächst.

Akzeptanzkriterien: - Formular für Frage und Antwort in der Detailansicht - Karte erscheint sofort nach dem Speichern in der Liste - Fehler werden angezeigt, wenn Felder leer gelassen werden

Implementiert: 2026-05-19

Datei: [frontend/src/StudyMate.jsx](#) (Funktion `addCard` , `handleAddCard`)

US-021 — Karteikarte bearbeiten

Als Eigentümer möchte ich bestehende Karteikarten bearbeiten können, damit ich Fehler in Fragen oder Antworten korrigieren kann.

Akzeptanzkriterien: - Bearbeiten-Button bei jeder Karte sichtbar (nur für Eigentümer) - Inline-Formular mit vorausgefüllten Feldern - Änderungen werden sofort im State und in der UI reflektiert

Implementiert: 2026-05-19 (Commit: `c2b967a feat: add card editing`)

Datei: [frontend/src/StudyMate.jsx](#) (Funktion `openEditCard` , `saveEditCard` , `handleEditCard`)

US-022 — Karteikarten per JSON importieren

Als Benutzer möchte ich mehrere Karteikarten auf einmal per JSON importieren können, damit ich schnell große Lernsets aufbauen kann.

Akzeptanzkriterien: - Import-Funktion in der Detailansicht (nur für Eigentümer) - JSON-Format: Array von Objekten mit `question` und `answer` - Ungültige Eingaben erzeugen eine verständliche Fehlermeldung - Nach Erfolg werden importierte Karten sofort angezeigt

Implementiert: 2026-05-26 (Commit: `9282ff3`)

Datei: [frontend/src/StudyMate.jsx](#) (Funktion `importCards` , `handleImportCards`)

US-023 — Karteikarte löschen

Als Eigentümer möchte ich einzelne Karteikarten löschen können, damit ich veraltete oder fehlerhafte Karten entfernen kann.

Akzeptanzkriterien: - Löschen-Button bei jeder Karte (nur für Eigentümer) - Bestätigungsdialo**g** vor dem Löschen - Karte verschwindet sofort aus der Ansicht

Implementiert: 2026-05-19

Datei: [frontend/src/StudyMate.jsx](#) (Funktion `deleteCard` , `handleDeleteCard`)

US-024 — Karten-Reihenfolge ändern

Als Eigentümer eines Sets möchte ich die Reihenfolge der Karteikarten anpassen können, damit ich thematisch verwandte Karten gruppieren kann.

Akzeptanzkriterien: - Pfeil-Buttons (▲/▼) neben jeder Karte (nur für Eigentümer) - Karte tauscht Position mit dem Nachbar oberhalb/unterhalb - Neue Reihenfolge wird dauerhaft in der Datenbank gespeichert (`position` -Feld) - Lern- und Quizmodus verwenden die aktualisierte Reihenfolge

Implementiert: 2026-06-02

Dateien: [frontend/src/StudyMate.jsx](#) (Funktion `handleMoveCard`),
[frontend/src/components/DetailView.jsx](#) (Buttons `ChevronUp` / `ChevronDown`)

Epic 4: Lernen und Quiz

US-030 — Lernmodus

Als Benutzer möchte ich meine Karteikarten im Flip-Card-Format durchgehen können, damit ich den Lernfortschritt tracken und mich selbst testen kann.

Akzeptanzkriterien: - Karten werden nacheinander angezeigt (Vorderseite = Frage) - Klick auf Karte dreht sie um und zeigt die Antwort - Nach Aufdecken kann der Benutzer „Gewusst“ oder „Nicht gewusst“ angeben - Fortschrittsbalken zeigt aktuelle Position - Am Ende erscheint eine Zusammenfassung mit gewussten/nicht-gewussten Karten - Option, die Session zu wiederholen

Implementiert: 2026-05-19

Datei: [frontend/src/StudyMate.jsx](#) (Komponente `LearnView`)

US-031 — Streak-Tracking

Als Benutzer möchte ich meinen täglichen Lern-Streak sehen können, damit ich motiviert bleibe, jeden Tag zu lernen.

Akzeptanzkriterien: - Streak wird im Dashboard und im Profil angezeigt - Streak erhöht sich nur einmal pro Kalendertag nach abgeschlossener Lernsession - Wird an einem Tag nicht gelernt, setzt sich der Streak auf 0 zurück - Daten werden in localStorage gespeichert (persistiert über Browser-Sessions)

Implementiert: 2026-05-26 (Commit: `28cf558 feat: enhance streak tracking`)

Datei: [frontend/src/StudyMate.jsx](#) (Funktionen `awardDailyStreak` , `getStreakState` , `toISODate` , `getDaysSince`)

US-032 — Standard-Quiz

Als Benutzer möchte ich ein Multiple-Choice-Quiz aus meinen Karteikarten starten können, damit ich mein Wissen testen kann.

Akzeptanzkriterien: - Quiz ist verfügbar, wenn das Set mindestens 4 Karten enthält - Falsche Antworten werden aus anderen Karten des Sets generiert - Nach der Auswahl wird die richtige Antwort grün, die

falsche rot markiert - Am Ende erscheint die Auswertung (richtig/falsch)

Implementiert: 2026-05-19

Datei: [frontend/src/StudyMate.jsx](#) (Komponente `QuizView` , Phase `quiz`)

US-033 — KI-Quiz

Als Benutzer möchte ich ein KI-generiertes Quiz starten können, damit ich mit neuartigen, thematisch passenden Fragen geübt werde.

Akzeptanzkriterien: - KI-Quiz wird über den Backend-Endpoint `/quiz/generate` generiert - Jede Frage enthält 4 Antwortoptionen (3 falsche, 1 richtige) und eine Erklärung - Nach der Antwort wird die Erklärung angezeigt - Ist kein Groq-API-Key konfiguriert, erscheint eine Hinweismeldung (HTTP 503)

Implementiert: 2026-05-19 (Commit: `db4dcbb` , initial mit Google Gemini); auf Groq (Llama) umgestellt: 2026-06-01 (Commit: `5e42b21 feat: update API keys and dependencies for quiz generation functionality`)

Dateien: [backend/app/routers/quiz.py](#), [frontend/src/StudyMate.jsx](#) (Funktion `generateAIQuiz` , Phase `aiquiz`)

Epic 5: Entdecken und soziale Funktionen

US-040 — Öffentliche Sets entdecken

Als Benutzer möchte ich öffentliche Sets anderer Benutzer entdecken und durchsuchen können, damit ich von der Lernarbeit anderer profitieren kann.

Akzeptanzkriterien: - „Entdecken“-Tab im Dashboard zeigt alle öffentlichen Sets - Suchfeld filtert Sets nach Titel in Echtzeit - Vorgeschlagene Sets werden am Anfang der Liste angezeigt

Implementiert: 2026-05-19

Datei: [frontend/src/StudyMate.jsx](#) (Komponente `DashboardView` , Tab `discover`)

US-041 — Favoriten

Als Benutzer möchte ich Sets als Favoriten markieren können, damit ich schnell auf häufig genutzte Sets zugreifen kann.

Akzeptanzkriterien: - Stern-Button bei jedem Set (aktiv = gelb) - Favoriten werden im Browser-localStorage gespeichert (`sm_favs_{userId}` bzw. `sm_favs_guest` für Gäste) - Favoritenansicht zeigt ausschließlich markierte Sets (auch für Gäste sichtbar) - Beim Einloggen werden Gastfavoriten automatisch mit den Benutzerfavoriten zusammengeführt

Implementiert: 2026-05-19 ; Gastfavoriten-Migration: 2026-06-02

Datei: [frontend/src/StudyMate.jsx](#) (Funktion `toggleFavorite` , Komponente `FavoritesView`)

US-042 — Freunde finden und verwalten

Als Benutzer möchte ich andere Benutzer suchen und als Freund hinzufügen können, damit ich mich mit ihnen vernetzen kann.

Akzeptanzkriterien: - Suche nach Benutzername oder Anzeigename in der Freunde-Ansicht - Freundschaftsanfrage senden, annehmen oder ablehnen - Bestehende Freundschaften können entfernt werden - Eingehende Anfragen werden in einem eigenen Tab mit Badge-Zähler in der Sidebar angezeigt

Implementiert: 2026-06-02 (Commit: `a15800c feat: implement friends feature with friend requests and public profiles`)

Datei: [frontend/src/components/FriendsView.jsx](#), [frontend/src/StudyMate.jsx](#) (Funktionen `handleSendFriendRequest` , `handleAcceptFriend` , `handleDeclineFriend` , `handleRemoveFriend` , `handleSearchUsers`)

US-043 — Öffentliches Profil ansehen

Als Benutzer möchte ich das öffentliche Profil eines anderen Benutzers ansehen können, damit ich seine öffentlichen Sets entdecken und ihn als Freund hinzufügen kann.

Akzeptanzkriterien: - Aufrufbar über den Autorennamen eines öffentlichen Sets (sofern `show_author` aktiv) oder aus der Freundesliste - Zeigt Avatar, Anzeigename, Bio und öffentliche Sets des Benutzers - Zeigt den aktuellen Freundschaftsstatus mit passender Aktion (Anfrage senden / Annehmen / Ablehnen / Entfernen) - Zurück-Button führt zur ursprünglichen Ansicht zurück (Dashboard oder Freundesliste)

Implementiert: 2026-06-02 (Commit: `a15800c`)

Datei: [frontend/src/components/PublicProfileView.jsx](#), [frontend/src/StudyMate.jsx](#) (Funktion `handleOpenUserProfile`)

Epic 6: Onboarding und UX

US-050 — Geführte Tour

Als neuer Benutzer möchte ich beim ersten Login eine geführte Tour durch die wichtigsten Funktionen erhalten, damit ich mich schnell zurechtfinde.

Akzeptanzkriterien: - Tour startet automatisch beim ersten Dashboard-Besuch nach dem Login - Tour startet automatisch beim ersten Öffnen einer Set-Detailansicht - Tour startet automatisch beim ersten Öffnen des „Set erstellen“-Dialogs - Jeder Tour-Schritt hebt das relevante UI-Element hervor und zeigt eine Erklärung - Die Tour kann jederzeit übersprungen werden - Abgeschlossene Tours werden in localStorage gespeichert (kein erneuter Autostart) - Ein Hilfe-Button in der Navbar ermöglicht das manuelle Neustarten der Tour

Implementiert: 2026-05-26 (Commit: 322ab03); Detail- & Erstellungstour: 2026-06-02

Datei: [frontend/src/StudyMate.jsx](#) (Komponente `GuidedTourOverlay`, Konstante `TOUR_STEPS`)

US-051 — Light/Dark-Mode

Als Benutzer möchte ich zwischen einem hellen und einem dunklen Farbschema wechseln können, damit ich je nach Umgebung komfortabel lernen kann.

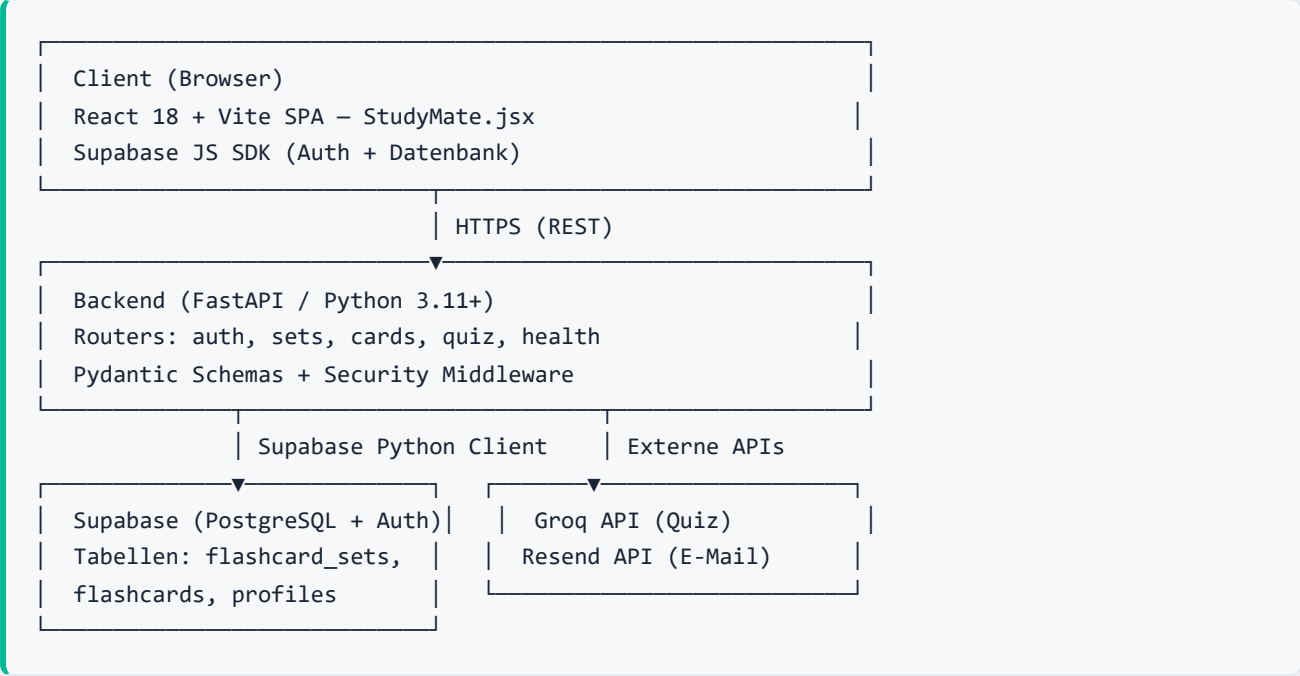
Akzeptanzkriterien: - Umschalter in der Navigationsleiste - Gewähltes Theme wird in localStorage gespeichert und beim nächsten Besuch wiederhergestellt - Alle UI-Elemente passen sich dem Theme an

Implementiert: 2026-05-26 (Commit: dcc0679 `Light/Darkmode`)

Datei: [frontend/src/StudyMate.jsx](#) (State `theme`, CSS-Klasse `.sm.light`)

3. Architekturübersicht

Schichtenarchitektur



Technologie-Stack

Schicht	Technologie	Version
Frontend Framework	React	18
Build Tool	Vite	aktuell
Styling	Inline CSS + dynamische Styles	—
Icon-Bibliothek	Lucide React	aktuell
Backend Framework	FastAPI	aktuell
Sprache Backend	Python	3.11+
Validierung	Pydantic v2	aktuell
Datenbank & Auth	Supabase (PostgreSQL)	aktuell
E-Mail	Resend API	—
KI-Integration	Groq (Llama 3.1 8B Instant, OpenAI-kompatible API)	—
Deployment	Vercel	—

4. Projektstruktur

```
studymate_v2/
├── documentation/
│   └── full_documentation_de.md    ← Diese Datei
├── studymate_v2/
│   ├── frontend/
│   │   ├── src/
│   │   │   ├── StudyMate.jsx      ← Gesamte App-Logik und alle Views
│   │   │   ├── main.jsx           ← React-Einstiegspunkt
│   │   │   └── supabase.js        ← Supabase-Client-Konfiguration
│   │   ├── index.html
│   │   ├── package.json
│   │   ├── vite.config.js
│   │   └── vercel.json
│   ├── backend/
│   │   ├── app/
│   │   │   ├── main.py            ← FastAPI App-Initialisierung
│   │   │   ├── core/
│   │   │   │   ├── config.py      ← Pydantic Settings
│   │   │   │   └── security.py    ← Token-Validierung, Eigentümer-Prüfung
│   │   │   ├── db/
│   │   │   │   └── supabase_client.py ← Supabase Admin/Public Clients
│   │   │   ├── routers/
│   │   │   │   ├── auth.py        ← Passwort-Reset-Endpunkt
│   │   │   │   ├── sets.py        ← CRUD Lernsets + Fork
│   │   │   │   ├── cards.py       ← CRUD Karteikarten
│   │   │   │   ├── quiz.py        ← KI-Quiz-Generierung
│   │   │   │   └── health.py       ← Health-Check
│   │   │   └── schemas/
│   │   │       ├── sets.py        ← StudySetCreate, StudySetUpdate
│   │   │       ├── cards.py       ← FlashcardCreate
│   │   │       ├── quiz.py        ← QuizGenerateRequest/Response
│   │   │       └── auth.py        ← ForgotPasswordRequest
│   │   ├── app.py                 ← Uvicorn-Einstiegspunkt
│   │   ├── requirements.txt
│   │   └── sql_schema.sql         ← Referenz-SQL für Tabellen
```

5. Datenbankschema

Das System nutzt **Supabase (PostgreSQL)** als Datenpersistenz-Schicht. Die Tabellen werden direkt über die Supabase-Dashboard-GUI oder via SQL angelegt.

Tabellen

flashcard_sets — Lernsets

Spalte	Typ	Beschreibung
id	uuid (PK)	Eindeutige ID, automatisch generiert
owneruserid	uuid (FK → auth.users)	Eigentümer des Sets
title	text	Titel des Sets (Pflichtfeld)
description	text	Beschreibung (optional)
ispublic	boolean	Öffentlich oder privat (Standard: <code>false</code>)
createdat	timestampz	Erstellungszeitpunkt
forked_from	uuid (FK → flashcard_sets, optional)	Verweist auf das Quell-Set, falls dieses Set durch Forken entstanden ist
show_author	boolean	Ob bei öffentlichen Sets der Ersteller (Avatar/Name, anklickbar zum Profil) angezeigt wird (Standard: <code>true</code>)

flashcards — Karteikarten

Spalte	Typ	Beschreibung
id	uuid (PK)	Eindeutige ID
setid	uuid (FK → flashcard_sets)	Zugehöriges Set
question	text	Frage (Pflichtfeld)
answer	text	Antwort (Pflichtfeld)
position	integer	Reihenfolge im Set (Sortierschlüssel für Lern-/Quiz-Modus)

profiles — Benutzerprofile

Spalte	Typ	Beschreibung
id	uuid (PK, FK → auth.users)	Supabase Auth User ID
username	text	Benutzername (eindeutig, lowercase)
displayname	text	Anzeigename
recovery_email	text	E-Mail für Passwort-Reset (optional)
bio	text	Kurzbeschreibung im Profil (optional)
image_data	text	Profilbild, Base64-kodiert (optional)
streak_count	integer	Aktueller Lern-Streak (Tage in Folge)
streak_last_date	text	Datum (ISO) des letzten Streak-Updates
activity_data	jsonb	Tägliche Lernaktivität für die Wochenansicht (<code>WeeklyActivityChart</code>)

friendships — Freundschaften

Spalte	Typ	Beschreibung
id	uuid (PK)	Eindeutige ID
user_id	uuid (FK → auth.users)	Anfragender Benutzer
friend_id	uuid (FK → auth.users)	Angefragter Benutzer
status	text	<code>pending</code> (Anfrage offen) oder <code>accepted</code> (befreundet)

Hinweis: `profiles` und `friendships` werden ausschließlich direkt über den Supabase JS Client im Frontend gelesen/geschrieben (kein eigener Backend-Endpunkt) — anders als `flashcard_sets` / `flashcards`, die über die FastAPI-Routen `sets.py` / `cards.py` laufen. Beide Tabellen sowie die Spalten `forked_from` und `show_author` fehlen aktuell in `sql_schema.sql` (siehe unten); das Referenzschema wurde nicht synchron zur Live-Datenbank gepflegt.

SQL-Referenzschema (aus `sql_schema.sql`)

```
CREATE TABLE IF NOT EXISTS public.study_sets (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  owner_id    UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  title       TEXT NOT NULL,  
  description TEXT,  
  is_public   BOOLEAN NOT NULL DEFAULT FALSE,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);  
  
CREATE TABLE IF NOT EXISTS public.flashcards (  
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  set_id      UUID NOT NULL REFERENCES public.study_sets(id) ON DELETE CASCADE,  
  question    TEXT NOT NULL,  
  answer      TEXT NOT NULL,  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Hinweis: In der produktiven Supabase-Datenbank weichen die Spaltennamen leicht ab (z. B. `owneruserid` statt `owner_id`, `ispublic` statt `is_public`). Die Datei `sql_schema.sql` dient als Referenz.

6. Backend — Komponenten und API-Referenz

6.1 `app/main.py` — Anwendungsinitialisierung

Erstellt die FastAPI-Anwendung, registriert CORS-Middleware und bindet alle Router ein.

```
app = FastAPI(title="StudyMate API", version="0.1.0")  
  
app.add_middleware(CORSMiddleware, allow_origins=["*"], ...)  
  
app.include_router(health.router)  
app.include_router(auth.router, prefix="/auth")  
app.include_router(sets.router, prefix="/sets")  
app.include_router(cards.router)  
app.include_router(quiz.router)
```

Implementiert: 2026-05-19

6.2 app/core/config.py — Konfiguration

Verwendet `pydantic-settings` für typsichere Konfiguration aus Umgebungsvariablen.

```
class Settings(BaseSettings):
    supabase_url: str = ""
    supabase_anon_key: str = ""
    supabase_service_role_key: str = ""
    resend_api_key: str = ""
    resend_from: str = "StudyMate <onboarding@resend.dev>"
    groq_api_key: str = ""
```

6.3 app/core/security.py — Authentifizierung und Autorisierung

Stellt FastAPI-Dependency-Funktionen bereit:

Funktion	Beschreibung
<code>get_current_user_id(token)</code>	Validiert Bearer-Token via Supabase, gibt User-ID zurück
<code>get_current_user(token)</code>	Wie oben, gibt vollständiges User-Objekt zurück
<code>require_set_owner(set_id, user_id)</code>	Prüft, ob der User Eigentümer des Sets ist (HTTP 403 sonst)
<code>require_set_access(set_id, user_id)</code>	Erlaubt Zugriff, wenn Set öffentlich oder User Eigentümer ist
<code>require_card_owner(card_id, user_id)</code>	Prüft Eigentümerschaft über Set der Karte

Implementiert: 2026-05-19

6.4 API-Endpunkte — Vollständige Referenz

Auth (/auth)

Methode	Pfad	Auth	Beschreibung
POST	<code>/auth/forgot-password</code>	Nein	Sendet 6-stelligen Passwort-Reset-Code per E-Mail

Request: POST `/auth/forgot-password`

```
{ "username": "max_mustermann" }
```

Response: `{ "message": "ok" }` (immer, unabhängig ob Benutzer existiert — Schutz vor Username-Enumeration)

Implementierungsdetail: Der Backend-Router sucht die Recovery-E-Mail aus der `profiles` -Tabelle. Falls vorhanden, wird über die Supabase Admin API (`generate_link` , Typ `recovery`) ein `email_otp` -Code erzeugt und via Resend versandt. Das Frontend ruft anschließend direkt `supabase.auth.verifyOtp({ email, token, type: "recovery" })` auf, um eine Session zu erhalten, und setzt darüber das neue Passwort via `supabase.auth.updateUser()` . Es gibt **keinen** klickbaren Link und keinen Redirect — der Code wird manuell im Formular eingegeben.

Sets (`/sets`)

Methode	Pfad	Auth	Beschreibung
GET	<code>/sets/public</code>	Nein	Alle öffentlichen Sets
GET	<code>/sets/my</code>	Ja	Eigene Sets des Benutzers
POST	<code>/sets</code>	Ja	Neues Set erstellen
PUT	<code>/sets/{set_id}</code>	Ja (Eigentümer)	Set aktualisieren
DELETE	<code>/sets/{set_id}</code>	Ja (Eigentümer)	Set löschen
POST	<code>/sets/{set_id}/fork</code>	Ja	Öffentliches Set forken

Request-Schema `StudySetCreate` :

```
{
  "title": "Cybersecurity Grundlagen",
  "description": "Wichtige Begriffe und Konzepte",
  "is_public": false
}
```

Fork-Endpunkt `POST /sets/{set_id}/fork` : - Prüft, ob das Quell-Set öffentlich ist (HTTP 403 sonst) - Erstellt eine Kopie des Sets mit `ispublic = false` - Kopiert alle Karteikarten in das neue Set - Gibt das neue Set mit Karten zurück (`select("*, flashcards(*)")`)

Implementiert: Sets: `2026-05-19` , Fork: `2026-05-26`

Cards (`/sets/{set_id}/cards` , `/cards/{card_id}`)

Methode	Pfad	Auth	Beschreibung
GET	<code>/sets/{set_id}/cards</code>	Optional	Karten eines Sets abrufen
POST	<code>/sets/{set_id}/cards</code>	Ja (Eigentümer)	Neue Karte erstellen
DELETE	<code>/cards/{card_id}</code>	Ja (Eigentümer)	Karte löschen

Besonderheit `GET /sets/{set_id}/cards` : - Öffentliche Sets sind ohne Auth abrufbar - Private Sets erfordern gültigen Bearer-Token des Eigentümers - HTTP 401 bei fehlendem Token, HTTP 403 bei falschem Benutzer

Request-Schema `FlashcardCreate` :

```
{
  "question": "Was ist SQL Injection?",
  "answer": "Eine Angriffstechnik, bei der SQL-Code in Eingabefelder eingeschleust wird.",
  "position": 1
}
```

Implementiert: 2026-05-19

Quiz (/quiz)

Methode	Pfad	Auth	Beschreibung
POST	/quiz/generate	Nein	KI-Quiz generieren (Groq/Llama)

Request-Schema `QuizGenerateRequest` :

```
{
  "cards": [
    { "q": "Was ist Phishing?", "a": "Betrugsversuch per gefälschten E-Mails" }
  ],
  "count": 5
}
```

Response-Schema `QuizGenerateResponse` :

```
{
  "questions": [
    {
      "question": "Was ist Phishing?",
      "options": ["Option A", "Option B", "Betrugsversuch per gefälschten E-Mails", "Option D"],
      "correct": 2,
      "explanation": "Phishing bezeichnet den Versuch, über gefälschte E-Mails oder Webseiten an"
    }
  ]
}
```

Implementierungsdetail: Die KI (Groq, Modell `llama-3.1-8b-instant` , angesprochen über die OpenAI-kompatible Chat-Completions-API unter <https://api.groq.com/openai/v1>) generiert

ausschließlich die drei falschen Antwortoptionen und eine Erklärung. Die richtige Antwort kommt direkt aus der Datenbank und wird an einer zufälligen Position eingefügt.

Implementiert: 2026-05-26 (Commit: `cb47bd8`); KI-Anbieter auf Groq umgestellt: 2026-06-01 (Commit: `5e42b21`)

7. Frontend — Komponenten und Funktionen

Die gesamte Frontend-Logik befindet sich in einer einzigen Datei: `frontend/src/StudyMate.jsx`. Diese Entscheidung wurde bewusst getroffen, um die Komplexität der Projektkonfiguration zu minimieren.

7.1 Hilfsfunktionen

Funktion	Beschreibung	Implementiert
<code>accentFor(id)</code>	Deterministischer Farbhash aus der Set-ID	2026-05-19
<code>normalizeSet(raw)</code>	Transformiert Supabase-Rohdaten in App-Format	2026-05-19
<code>toISODate(date)</code>	Konvertiert Date-Objekt in <code>YYYY-MM-DD</code>	2026-05-26
<code>parseISODate(str)</code>	Parst <code>YYYY-MM-DD</code> sicher zu Date	2026-05-26
<code>getDaysSince(str)</code>	Berechnet Tage seit einem Datum	2026-05-26
<code>readStreakStore()</code>	Liest Streak-Daten aus <code>localStorage</code>	2026-05-26
<code>writeStreakStore(store)</code>	Schreibt Streak-Daten in <code>localStorage</code>	2026-05-26
<code>getStreakState(key)</code>	Gibt aktuellen Streak zurück; setzt auf 0 bei > 1 Tag Pause	2026-05-26
<code>awardDailyStreak(key)</code>	Erhöht Streak einmalig pro Kalendertag	2026-05-26
<code>readProfileSettings(id)</code>	Liest Profildetails (Bio, Bild) aus <code>localStorage</code>	2026-05-26
<code>writeProfileSettings(id, s)</code>	Schreibt Profildetails in <code>localStorage</code>	2026-05-26

7.2 Navigations-Zustände (view)

View-Name	Beschreibung	Komponente
auth	Login/Registrierung	AuthView
forgot	Passwort-Reset (Benutzername → Code + neues Passwort, 2 Schritte intern)	ForgotPasswordView
dashboard	Hauptansicht mit Tabs	DashboardView
detail	Set-Detailansicht	DetailView
learn	Lernmodus	LearnView
quiz	Quiz-Modus	QuizView
favorites	Favoritenansicht	FavoritesView
friends	Freundesliste, Anfragen, Benutzersuche	FriendsView
public_profile	Öffentliches Profil eines anderen Benutzers	PublicProfileView
profile	Profilansicht	ProfileView
profile_edit	Profil bearbeiten	ProfileEditView
settings	Einstellungen (Platzhalter)	SettingsView

7.3 Hauptkomponenten

NavBar — Navigationsleiste

Zeigt Logo, Tour-Button, Theme-Umschalter, Profil-Avatar und Logout-Button.

Props: `user` , `onHome` , `onLogout` , `onGoToLogin` , `theme` , `onToggleTheme` , `onProfile` , `onTourRestart` , `currentViewHasTour`

Zeigt im Avatar-Button das hochgeladene Profilbild (`user.imageData`) als `<img>`-Element — fällt auf den Anfangsbuchstaben zurück, wenn kein Bild vorhanden ist. Der Avatar-Container hat `overflow: hidden` , damit runde Bilder korrekt abgeschnitten werden.

Implementiert: 2026-05-19 ; Tour-Button: 2026-05-26 ; Profilbild-Anzeige: 2026-06-01 (cf603c0)

Sidebar — Seitennavigation

Collapsible Sidebar mit Links zu Dashboard, Meine Sets, Entdecken, Favoriten, Freunde. Auf Mobilgeräten als Overlay, auf Desktop als feste Leiste.

Die Fußzeile der Sidebar zeigt seit `2026-06-01` einen kleinen Avatar (Profilbild oder Anfangsbuchstabe) neben dem Benutzernamen — konsistent mit dem NavBar-Avatar.

Implementiert: `2026-05-19` ; Profilbild in Footer: `2026-06-01` (`cf603c0`)

`AuthView` — Authentifizierung

Tabs für Login und Registrierung. Validierung der Eingaben (Benutzername: min. 3 Zeichen, alphanumerisch; Passwort: min. 6 Zeichen). Recovery-E-Mail als optionales Feld bei Registrierung.

Implementiert: `2026-05-19`

`DashboardView` — Dashboard

Drei Tabs: Dashboard (Statistiken), Entdecken (öffentliche Sets), Meine Sets. Enthält Suchfeld, Statistikkarten (Sets, Karten, Streak), Vorschlagskarussell für öffentliche Sets und das `WeeklyActivityChart` (Lernaktivität der letzten 7 Tage, aus `profiles.activity_data`). Auf öffentlichen Set-Karten wird bei aktiviertem `show_author` der Ersteller mit Avatar/Name angezeigt und ist anklickbar (öffnet `public_profile`).

Implementiert: `2026-05-19` ; `WeeklyActivityChart` : `2026-06-01` , responsives Feintuning: `2026-06-02`

`DetailView` — Set-Detailansicht

Zeigt alle Karten eines Sets mit Nummern. Eigentümer können: - Karten hinzufügen (Inline-Formular) - Karten bearbeiten (Inline-Formular mit Vorausfüllung) - Karten löschen - Karten per JSON importieren - Set-Titel und -Beschreibung bearbeiten - Sichtbarkeit umschalten - Set löschen

Fremde Sets (öffentlich) zeigen den Fork-Button.

Verhaltensänderung (`cf603c0` , `2026-06-01`): Die „Lernen starten“- und „Quiz starten“-Buttons sind nicht mehr bei leeren Sets deaktiviert. Die zugehörige Hinweismeldung („Noch keine Karten – füge welche hinzu“) wurde entfernt. Der `LearnView-Guard` für leere Sets wurde ebenfalls entfernt — die Logik für leere Sets wird damit implizit durch die Set-Erstellung gesteuert.

Implementiert: `2026-05-19` ; Bearbeiten/Import: `2026-05-26` ; Button-Guard entfernt: `2026-06-01`

`LearnView` — Lernmodus

Flip-Card-Ansicht mit CSS-3D-Transformation. Benutzer markiert jede Karte als gewusst oder nicht gewusst. Am Ende: Zusammenfassung + Option zum Wiederholen. Beim Abschließen wird `handleCompleteSet()` aufgerufen, der den Streak aktualisiert.

Implementiert: 2026-05-19

QuizView — Quiz-Modus

Unterstützt zwei Modi: - **Standard-Quiz:** Multiple-Choice aus vorhandenen Karten (mind. 4 Karten erforderlich) - **KI-Quiz:** Groq-generierte Fragen (Llama 3.1) mit Erklärung nach jeder Antwort

Implementiert: Standard: 2026-05-19 ; KI: 2026-05-26

GuidedTourOverlay — Geführte Tour

Overlay-Komponente, die UI-Elemente per CSS-Highlight hervorhebt und Erklärungen in einem Popup anzeigt. Schritte sind in `TOUR_STEPS` pro View definiert.

```
const TOUR_STEPS = {
  dashboard: [...], // 4 Schritte
  discover: [...], // 3 Schritte
  mine: [...], // 2 Schritte
  detail: [...], // 3 Schritte
  createSet: [...], // 2 Schritte
  learn: [...], // 4 Schritte
  quiz: [...], // 3 Schritte
  favorites: [...], // 3 Schritte
  friends: [...], // 2 Schritte
  profile: [...], // 2 Schritte
};
```

Tour-Status wird in `localStorage` unter `sm_tour_completed` gespeichert.

Implementiert: 2026-05-26 (Commit: 322ab03)

ProfileView und ProfileEditView — Profil

ProfileView : Zeigt Avatar, Name, Bio, Statistiken (Karten, Sets, Streak), Vorschau der letzten Sets.

ProfileEditView : Formular zum Ändern von Anzeigename, Bio und Profilbild. Anzeigename, Bio und Profilbild (Base64) werden in Supabase (`profiles.displayname` , `profiles.bio` , `profiles.image_data`) aktualisiert. Aus einer früheren, rein lokalen Version existiert noch eine einmalige Migration: Beim Login werden alte `localStorage` -Profildaten (`readProfileSettings`) in die Supabase-Spalten übernommen, falls dort noch nichts hinterlegt ist.

Implementiert: 2026-05-26 (Commits: 3d4d234 , 184d550)

FriendsView — Freunde

Drei Tabs: Freunde (akzeptierte Freundschaften), Anfragen (eingehende Freundschaftsanfragen mit Annehmen/Ablehnen), Suchen (Benutzersuche nach Username/Anzeigename). Lädt und schreibt direkt über den Supabase-Client auf die Tabelle `friendships` (kein Backend-Endpunkt).

Implementiert: 2026-06-02 (Commit: `a15800c feat: implement friends feature with friend requests and public profiles`)

Datei: `frontend/src/components/FriendsView.jsx`, `frontend/src/StudyMate.jsx` (Funktionen `fetchFriends`, `handleSendFriendRequest`, `handleAcceptFriend`, `handleDeclineFriend`, `handleRemoveFriend`, `handleSearchUsers`)

PublicProfileView — Öffentliches Profil

Zeigt Avatar, Name, Bio und öffentliche Sets eines anderen Benutzers sowie den aktuellen Freundschaftsstatus mit passender Aktion (Anfrage senden/Annehmen/Ablehnen/Entfernen). Erreichbar über einen Klick auf den Autorennamen eines öffentlichen Sets (Dashboard) oder aus der Freundesliste. Merkt sich die aufrufende View (`publicProfileFrom`), um beim Zurück-Button korrekt zu navigieren.

Implementiert: 2026-06-02 (Commit: `a15800c`)

Datei: `frontend/src/components/PublicProfileView.jsx`, `frontend/src/StudyMate.jsx` (Funktionen `handleOpenUserProfile`, `getFriendStatus`, `getFriendshipId`)

8. Authentifizierung und Sicherheit

Auth-Mechanismus

Das System nutzt einen hybriden Ansatz:

1. **Frontend-Auth:** Supabase JavaScript SDK verwaltet Sessions. Benutzername wird in eine Fake-E-Mail konvertiert (`username@studymate.local`), da Supabase Auth E-Mail-Format erwartet.

```
const toFakeEmail = (username) => `${username.toLowerCase()}@studymate.local`;
```

2. **Backend-Auth:** Geschützte Endpoints erwarten einen `Bearer`-Token im Authorization-Header. Der Token wird via Supabase Public Client validiert.
3. **Session-Management:** `supabase.auth.onAuthStateChange` wird vor `getSession()` registriert, um Login/Logout-Events korrekt abzufangen.
4. **Passwort-Reset:** Läuft seit 2026-06-23 über `supabase.auth.verifyOtp()` (Code-Eingabe) statt über einen Redirect-Link — es ist kein URL-/Hash-Parsing beim App-Start mehr nötig.

Sicherheitsrichtlinien

Aspekt	Maßnahme
Username-Enumeration	Passwort-Reset gibt immer <code>{ "message": "ok" }</code> zurück
Service Role Key	Nur im Backend, nie im Frontend-Code
Token-Validierung	Jeder geschützte Endpoint prüft den Token über Supabase
Eigentümer-Prüfung	<code>require_set_owner()</code> / <code>require_card_owner()</code> bei Schreiboperationen
CORS	Konfiguriert in <code>CORSMiddleware</code> ; in Produktion auf Frontend-URL einschränken
Sensible Daten	<code>.env</code> -Dateien sind in <code>.gitignore</code> (Commit: <code>6d2b4ad</code>)

9. Feature-Dokumentation (chronologisch)

Phase 1 — Grundfunktionen (2026-05-19)

Datum	Commit	Feature
2026-05-19	<code>8cea6d6</code>	Supabase-Integration: Auth + Datenbankbindung
2026-05-19	<code>048da9b</code>	Passwort-Reset-Flow mit E-Mail via Resend
2026-05-19	<code>db4dcbb</code>	KI-Quiz-Integration (Google Gemini)
2026-05-19	<code>81b7561</code>	Flip-Card-Animation (Performance-Optimierung)
2026-05-19	<code>fa7c4df</code>	Kartensets erstellen
2026-05-19	<code>c2b967a</code>	Karten bearbeiten und importieren
2026-05-19	<code>7038f50</code>	Verlinkung zur Anmeldeseite im Gastmodus

Phase 2 — Erweiterungen (2026-05-26)

Datum	Commit	Feature
2026-05-26	28cf558	Streak-Tracking mit Datum-Parsing und Reset-Logik
2026-05-26	cb47bd8	Migration auf google-genai SDK; CORS-Fix
2026-05-26	dcc0679	Light/Dark-Mode
2026-05-26	9282ff3	Set-Name bearbeiten, Import-Funktion
2026-05-26	98d6cdf	Fork-Feature: Öffentliche Sets kopieren
2026-05-26	4a41540	Fork-UX: Dialog, bessere Ausrichtung
2026-05-26	409a1e3	Fork-Feature-Dokumentation
2026-05-26	322ab03	Geführte Tour mit localStorage und Restart-Button
2026-05-26	3d4d234	Profil-Overlay
2026-05-26	e46744c	<code>onForkSet</code> -Prop zur DetailView-Komponente

Phase 3 — Bugfixes und Verbesserungen (2026-05-26 — 2026-06-01)

Datum	Commit	Beschreibung
2026-05-26	b332a07	Favoriten nur für eingeloggte Nutzer anzeigen
2026-05-26	f0e33f0	Benutzer-Feedback bei leeren Kartensets (Quiz/Lernen)
2026-05-26	0657acb	Edit-Icon in StudyMate-Komponente
2026-05-26	1a10046	Supabase-Version in requirements.txt fixiert
2026-05-26	184d550	Profil-Bug behoben
2026-06-01	cf603c0	Profilbild in NavBar-Avatar und Sidebar-Footer; Learn/Quiz-Button-Guard bei leeren Sets entfernt; <code>handleAddCard</code> -State-Duplizierung gefixt

Phase 4 — Soziale Funktionen, KI-Anbieter-Wechsel, UI-Feinschliff (2026-06-01 — 2026-06-09)

Datum	Commit	Beschreibung
2026-06-01	5e42b21	KI-Anbieter von Google Gemini auf Groq (Llama 3.1) umgestellt — Bibliothek, Modellname und Env-Var (<code>GEMINI_API_KEY</code> → <code>GROQ_API_KEY</code>) geändert
2026-06-01	70bb322	Aktivitäts-Tracking + neue Komponente <code>WeeklyActivityChart</code> (Lernaktivität der letzten 7 Tage im Dashboard)
2026-06-01	a52e919	Eigenständige <code>Sidebar</code> -Komponente mit Navigation und Profilanzeige (vorher Teil von <code>StudyMate.jsx</code>)
2026-06-01	596b27b	Playwright-E2E-Testframework eingeführt (erste Testdateien)
2026-06-02	a15800c	Neues Freunde-Feature: Freundschaftsanfragen, öffentliche Profile (<code>FriendsView</code> , <code>PublicProfileView</code> , Tabelle <code>friendships</code>); Autoren-Sichtbarkeits-Dialog (<code>show_author</code>) beim Veröffentlichen eines Sets
2026-06-02	4ceec09	Neue <code>Avatar</code> -Komponente, Modal-Styling überarbeitet
2026-06-02 – 06-08	mehrere	UI-Feinschliff: Sidebar-Einklapp-Button, Light/Dark-Mode-Korrekturen, NavBar/Logo-Positionierung, Pfeil-Buttons-Layout in der Detailansicht
2026-06-08	fd41c1e	Globaler Exception-Handler im Backend; verbessertes Fehlerhandling bei KI-Quiz-Antworten
2026-06-08	06fea88	Hinweistext bei leerem Kartenset im Quiz-Modus
2026-06-08	5d401d4	Ladezustand für Freundesliste; geführte Tour um die Views <code>learn</code> , <code>quiz</code> , <code>friends</code> , <code>profile</code> erweitert
2026-06-09	a50d7d1	Set-Detailansicht: Karten-Update-Logik überarbeitet, Spalte <code>forked_from</code> zur Nachverfolgung des Quell-Sets bei Forks ergänzt
2026-06-09	18b6bcc , 4ddf635	<code>WeeklyActivityChart</code> : responsives Layout, Schriftgrößen-Skalierung, Farbschema für vergangene/zukünftige Aktivitätspunkte

Phase 5 — Sicherheits-Fix Passwort-Reset (2026-06-23)

Datum	Commit	Beschreibung
2026-06-23	—	Passwort-Reset von Magic-Link auf OTP-Code umgestellt. Grund: E-Mail-Sicherheitsscanner konsumierten den Einweg-Link-Token bereits vor dem Klick des Nutzers, wodurch der Reset clientseitig nie ankam (User landete ohne Session auf der normalen Startseite). <code>ResetPasswordView</code> und der <code>PASSWORD_RECOVERY</code> -Auth-Event-Handler in <code>StudyMate.jsx</code> wurden entfernt, <code>FRONTEND_URL</code> ist dadurch kein benötigter Konfigurationswert mehr.

10. Konfiguration und Umgebungsvariablen

Backend (`.env` in `backend/`)

Variable	Pflicht	Beschreibung
<code>SUPABASE_URL</code>	Ja	URL des Supabase-Projekts
<code>SUPABASE_ANON_KEY</code>	Ja	Supabase Anon/Public Key
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Ja	Supabase Service Role Key (nur Backend!)
<code>RESEND_API_KEY</code>	Empfohlen	Resend API Key für E-Mail-Versand
<code>RESEND_FROM</code>	Nein	Absenderadresse (Standard: <code>StudyMate <onboarding@resend.dev></code>)
<code>GROQ_API_KEY</code>	Optional	Groq API Key für KI-Quiz (Llama 3.1)

Frontend (`.env.local` in `frontend/`)

Variable	Pflicht	Beschreibung
<code>VITE_SUPABASE_URL</code>	Ja	Supabase URL (öffentlich)
<code>VITE_SUPABASE_ANON_KEY</code>	Ja	Supabase Anon Key (öffentlich)
<code>VITE_API_URL</code>	Ja	Backend-URL (z. B. <code>http://localhost:8000</code>)

Sicherheitshinweis: `SUPABASE_SERVICE_ROLE_KEY` darf **niemals** im Frontend verwendet werden. Er gewährt vollständigen Datenbankzugriff ohne RLS-Einschränkungen.

11. Lokale Entwicklungsumgebung

11.1 Backend starten

Voraussetzungen: Python 3.11+, pip

```
# 1. In den Backend-Ordner wechseln
cd studymate_v2/backend

# 2. Virtuelle Umgebung erstellen und aktivieren
python -m venv .venv
source .venv/bin/activate      # macOS / Linux
# .venv\Scripts\activate      # Windows

# 3. Abhängigkeiten installieren
pip install -r requirements.txt

# 4. Umgebungsvariablen konfigurieren
cp .env.local.example .env.local # Vorlage kopieren
# Datei .env.local mit eigenen Werten befüllen

# 5. Server starten
uvicorn app.main:app --reload --port 8000
```

API-Dokumentation verfügbar unter: <http://localhost:8000/docs> (Swagger UI)

11.2 Frontend starten

Voraussetzungen: Node.js 18+, npm

```
# 1. In den Frontend-Ordner wechseln
cd studymate_v2/frontend

# 2. Abhängigkeiten installieren
npm install

# 3. Umgebungsvariablen konfigurieren
# .env.local mit VITE_SUPABASE_URL, VITE_SUPABASE_ANON_KEY, VITE_API_URL befüllen

# 4. Entwicklungsserver starten
npm run dev
# Läuft standardmäßig auf http://localhost:5173
```


12. UI/UX — Design und Animationen

Design-System

StudyMate verwendet ein konsistentes Design-System, das vollständig in `StudyMate.jsx` als injiziertes CSS definiert ist.

Farbpalette	Wert	Verwendung
Primärfarbe	<code>#00d4aa</code>	Buttons, Akzente, aktive Zustände
Hintergrund (Dark)	<code>#080c18</code>	App-Hintergrund
Text (primär)	<code>#f1f5f9</code>	Überschriften
Text (sekundär)	<code>#64748b</code>	Untertitel, Metadaten
Akzent violett	<code>#8b5cf6</code>	Quiz, KI-Features
Fehlerfarbe	<code>#ef4444</code>	Fehler, Löschen

Schriftarten: Sora (Interface), JetBrains Mono (Code, Zahlen)

Flip-Card-Animation

Die Lernkarten verwenden CSS-3D-Transformation. Wichtige CSS-Eigenschaften:

```
.sm-flip-scene    { perspective: 1200px; }
.sm-flip-card    { transform-style: preserve-3d; transition: transform .55s cubic-bezier(.4,0,
.sm-flip-card.flipped { transform: rotateY(180deg); }
.sm-flip-face    { backface-visibility: hidden; -webkit-backface-visibility: hidden; }
.sm-flip-back    { transform: rotateY(180deg) translateZ(1px); }
```

Bugfix-Hinweis (Firefox): `backface-visibility: hidden` muss auf **beiden** Seiten der Karte gesetzt sein, sonst kann in Firefox die Rückseite durchscheinen.

Responsive Design

- Desktop ($\geq 880\text{px}$): Sidebar dauerhaft sichtbar, Content verschoben (`margin-left: 260px`)
- Mobil ($< 880\text{px}$): Hamburger-Menü öffnet Sidebar als Overlay

13. CI/CD und Deployment

Deployment-Konfiguration

Beide Services können auf **Vercel** deployed werden:

Frontend `vercel.json` :

```
{
  "rewrites": [{ "source": "/(.*)", "destination": "/" }]
}
```

(SPA-Routing: alle Pfade werden auf `index.html` umgeleitet)

Backend `vercel.json` : Konfiguriert das Python-Backend als Vercel Serverless Functions.

Empfohlener CI/CD-Workflow

1. Push to Feature Branch
2. Linting (ESLint, flake8/ruff)
3. Unit Tests (pytest, vitest)
4. Build Frontend (npm run build)
5. Deploy to Staging
6. PR Review + Merge to main
7. Deploy to Production

Branching-Strategie (Aktuell)

Branch	Beschreibung
<code>main</code>	Produktionsstand
<code>devi</code>	Persönlicher Arbeitsbranch (Devi)
<code>Selina</code>	Persönlicher Arbeitsbranch (Selina)
<code>Jenny</code>	Persönlicher Arbeitsbranch (Jenny)
<code>Michelle</code>	Persönlicher Arbeitsbranch (Michelle)

Pull Requests werden nach Code-Review in `main` gemerged.

14. Tests und Qualitätssicherung

Aktueller Stand

Die Qualitätssicherung erfolgt durch zwei Ebenen: strukturiertes manuelles Testen (Projektboard) sowie automatisierte E2E-Tests mit Playwright, die alle manuell geprüften Szenarien reproduzierbar abdecken.

Durchgeführte manuelle Tests

Alle Tests wurden manuell im Browser (Chrome und Firefox) sowie auf mobilen Viewports (Responsive Design) durchgeführt. Die Ergebnisse wurden im Projektboard (Spalte „Review / Testing“) nachverfolgt.

Authentifizierung

Testfall	Ergebnis
Login mit gültigen Zugangsdaten	✓ Erfolgreich
Logout	✓ Session wird korrekt beendet
Gastansicht (ohne Login)	✓ Eingeschränkter Zugriff korrekt

Karteikarten & Lernsets

Testfall	Ergebnis
Karten erstellen	✓ Karte wird gespeichert und angezeigt
Karten laden	✓ Karten werden korrekt aus der Datenbank geladen
Karten bearbeiten	✓ Änderungen werden übernommen
Karten löschen	✓ Karte wird entfernt
Lernsets bearbeiten (Titel, Beschreibung)	✓ Erfolgreich
Lernsets löschen	✓ Erfolgreich, Set verschwindet aus der Liste
Private Lernsets	✓ Nur für Eigentümer sichtbar

Infrastruktur & Deployment

Testfall	Ergebnis
Supabase-Verbindung prüfen	✓ Datenbankabfragen funktionieren korrekt
Deployment (Vercel/Production)	✓ Produktivumgebung erreichbar und funktionsfähig
Mobile Ansicht	✓ Layout passt sich korrekt an kleine Bildschirme an

Gamification

Testfall	Ergebnis
Streaks	✅ Lernserie wird korrekt gezählt und angezeigt
Guided Tour	✅ Tour startet bei Erstnutzung und kann übersprungen werden

Bekannte Bugs (identifiziert beim Testen)

Bug	Status
Sterne-Favoriten-Icon überlappt den „Karten“-Text in der Set-Karte	Offen
Startseite leitet in Firefox nicht auf das Dashboard weiter	Offen

Bugfix: Fork-Funktion kopiert Karten ohne Position (2026-06-01)

Problem: Beim Forken eines öffentlichen Sets wurden die Karteikarten ohne das `position`-Feld in die neue Kopie eingefügt. Dadurch verloren die Karten ihre Reihenfolge, da `GET /sets/{set_id}/cards` nach `position` sortiert.

Ursache: In `backend/app/routers/sets.py` fehlte `"position": card.get("position", 0)` beim Einfügen der kopierten Karten. Außerdem wurden die Quellkarten nicht nach `position` abgefragt.

Fix: Quellkarten werden nun mit `.order("position")` abgefragt, und das `position`-Feld wird beim Insert in die Kopie übernommen.

Getestet durch: Manuellen Test — öffentliches Set mit mehreren geordneten Karten geforkt, Reihenfolge der Karten im geforkten Set überprüft.

Automatisierte E2E-Tests mit Playwright (ab 2026-06-01)

Die E2E-Tests befinden sich im Verzeichnis `frontend/e2e/` und laufen mit [Playwright](#). Sie decken alle manuell geprüften Szenarien ab und führen sie automatisiert gegen die echte Supabase-Instanz aus.

Voraussetzungen

1. Einen Test-Benutzer in Supabase anlegen (einmalig über die App registrieren oder über das Supabase-Dashboard)
2. Zugangsdaten in `frontend/.env.test` eintragen:

```
TEST_USERNAME=testUser
TEST_PASSWORD=password
```

Tests ausführen

```
cd studymate_v2/frontend

# Alle Tests (Desktop Chrome + Mobile Chrome)
npm run test:e2e

# Interaktiver UI-Modus
npm run test:e2e:ui

# HTML-Bericht anzeigen
npm run test:e2e:report
```

Testdateien

Datei	Abgedeckte Szenarien
<code>e2e/auth.spec.js</code>	Login, Registrierung, Logout, Gastansicht (4 Beschreibungen, 9 Tests)
<code>e2e/sets.spec.js</code>	Supabase-Verbindung, Set erstellen/bearbeiten/löschen, Private Sets (4 Beschreibungen + 1 eigenständiger Test, 7 Tests)
<code>e2e/cards.spec.js</code>	Karten laden, erstellen, bearbeiten, löschen (3 Beschreibungen + 1 eigenständiger Test, 4 Tests)
<code>e2e/ui.spec.js</code>	Guided Tour, Streaks (2 Beschreibungen, 5 Tests)
<code>e2e/mobile.spec.js</code>	Mobile Ansicht auf Pixel-5-Viewport (1 Beschreibung, 3 Tests)
<code>e2e/features.spec.js</code>	Registrierung, Karten-Reihenfolge ändern, Set kopieren (Priv-zu-Priv), Gastfavoriten, KI-Quiz-Fehlerbehandlung, Geführte Tour (Detailansicht + Set-Erstellung) (7 Beschreibungen, 11 Tests)

Konfiguration

`frontend/playwright.config.js` definiert zwei Projekte:

- **Desktop Chrome** — läuft alle Tests außer `mobile.spec.js`
- **Mobile Chrome (Pixel 5)** — läuft nur `mobile.spec.js`

Der Dev-Server wird beim Start automatisch hochgefahren (`reuseExistingServer: true` wiederverwendet einen laufenden Server).

Wichtige Hinweise

- Tests verwenden Timestamps-basierte eindeutige Titel (`PW_Set_<uid>`), um Testisolation zu gewährleisten
- Erstellte Testdaten werden am Ende jedes Tests gelöscht (Cleanup in `afterEach` bzw. am Testende)
- `.env.test` ist in `.gitignore` eingetragen und wird nicht ins Repository eingcheckedt

Empfohlene Backend-Tests (zukünftig, pytest)

```
pip install pytest pytest-asyncio httpx
pytest -q
```

Beispiel-Testfälle:

```
def test_get_public_sets(client):
    response = client.get("/sets/public")
    assert response.status_code == 200
    assert isinstance(response.json(), list)

def test_fork_private_set_returns_403(client, auth_headers):
    response = client.post("/sets/private-set-id/fork", headers=auth_headers)
    assert response.status_code == 403

def test_fork_preserves_card_order(client, auth_headers, public_set_with_ordered_cards):
    response = client.post(f"/sets/{public_set_with_ordered_cards}/fork", headers=auth_headers)
    assert response.status_code == 200
    cards = response.json()["flashcards"]
    positions = [c["position"] for c in cards]
    assert positions == sorted(positions)
```

15. Troubleshooting und FAQ

Problem: CORS-Fehler beim API-Aufruf

Ursache: Backend-URL falsch oder CORS nicht konfiguriert.

Lösung: `VITE_API_URL` im Frontend und `CORS_ORIGINS` im Backend prüfen. Backend neu starten nach `.env`-Änderung.

Problem: Supabase Token-Fehler (401)

Ursache: Abgelaufene Session oder falsche Keys.

Lösung: `SUPABASE_URL` und `SUPABASE_ANON_KEY` prüfen.

Problem: Passwort-Reset-Code wird abgelehnt ("Code ungültig oder abgelaufen")

Ursache: Code falsch eingegeben, älter als 1 Stunde, oder bereits einmal verwendet (jeder Code ist Einweg).

Lösung: Neuen Code über „Passwort vergessen?“ anfordern. Bei wiederholtem Auftreten: Supabase Dashboard → Authentication → Logs prüfen, ob der Code z. B. durch einen automatisierten Request (Mail-Scanner) bereits verbraucht wurde, bevor der Nutzer ihn eingegeben hat.

Problem: Flip-Card zeigt Vor- und Rückseite gleichzeitig (Firefox)

Lösung: CSS aus Abschnitt 12 verwenden. Sicherstellen, dass `backface-visibility: hidden` auf **beiden** `.sm-flip-face` -Elementen gesetzt ist.

Problem: KI-Quiz schlägt fehl

Ursache: `GROQ_API_KEY` nicht konfiguriert.

Lösung: API-Key in Backend `.env` setzen. Die Antwort `503 KI nicht konfiguriert.` bestätigt fehlendes Key.

Problem: Passwort-Reset E-Mail kommt nicht an

Ursache: `RESEND_API_KEY` fehlt oder Recovery-E-Mail im Profil nicht hinterlegt.

Lösung: Key in Backend `.env` setzen. Benutzer muss bei Registrierung eine Recovery-E-Mail angegeben haben.

Problem: Fork-Funktion gibt Fehler zurück

Ursache: Benutzer ist nicht eingeloggt oder versucht ein privates Set zu forken.

Lösung: Nur öffentliche Sets können geforkt werden. Authentifizierungstoken muss gültig sein.

16. ChangeLog

Version 0.3.2 (2026-06-23) — Aktuelle Version

Sicherheits-Fix: - Passwort-Reset von klickbarem Magic-Link auf manuell eingegebenen 6-stelligen OTP-Code umgestellt (`backend/app/routers/auth.py` , `frontend/src/components/AuthViews.jsx`). Ursache war, dass E-Mail-Sicherheitsscanner den Einweg-Token des Links vor dem eigentlichen Klick des Nutzers verbrauchten — der Reset landete dadurch nie auf einer gültigen Session. - `ResetPasswordView` und der `PASSWORD_RECOVERY` -Event-Handler (`onAuthStateChange`) in `StudyMate.jsx` entfernt, da nicht mehr benötigt. - Konfigurationsvariable `FRONTEND_URL` entfernt (war nur für den Redirect des alten Magic-Links nötig).

Aktualisierte Dokumentationsabschnitte: US-004, Abschnitt 6.2, 6.4, 7.2, 8, 10, 15, ChangeLog

Version 0.3.1 (2026-06-09)

Schließt die Dokumentationslücke zwischen `cf603c0` (2026-06-02) und `18b6bcc` (2026-06-09) auf `main` — dieser Stand war zuvor nicht dokumentiert.

Neue Funktionen: - Freunde-Feature: Freundschaftsanfragen senden/annehmen/ablehnen/entfernen, Benutzersuche, öffentliche Profile (`FriendsView` , `PublicProfileView` , neue Tabelle `friendships`) — US-042, US-043 - Autoren-Sichtbarkeits-Dialog (`show_author`) beim Veröffentlichen eines Sets - `WeeklyActivityChart` -Komponente im Dashboard (Lernaktivität der letzten 7 Tage aus `profiles.activity_data`), inkl. responsivem Feinschliff und Farbschema für vergangene/zukünftige Tage - Eigenständige `Sidebar` -Komponente (Navigation + Profilanzeige), neue `Avatar` -Komponente - Geführte Tour um die Views `learn` , `quiz` , `friends` , `profile` erweitert (vorher nur `dashboard` , `discover` , `mine` , `detail` , `createSet`) - Spalte `forked_from` auf `flashcard_sets` zur Nachverfolgung des Quell-Sets bei Forks - Globaler Exception-Handler im Backend; Hinweistext bei leerem Kartenset im Quiz-Modus

Breaking Change: - KI-Anbieter von Google Gemini auf Groq (Llama 3.1 8B Instant) umgestellt — Env-Var `GEMINI_API_KEY` existiert nicht mehr, ersetzt durch `GROQ_API_KEY` ; Bibliothek `google-genai` durch `openai` (mit Groqs OpenAI-kompatiblen Endpunkt) ersetzt

Aktualisierte Dokumentationsabschnitte: US-012, US-042, US-043, Abschnitt 3, 5, 6.2, 6.4, 7.2, 7.3, 9, 10, 14, 15, ChangeLog

Version 0.3.0 (2026-06-02)

Neue Funktionen: - Registrierung erstellt jetzt explizit einen `profiles` -Eintrag (Benutzername-Eindeutigkeit wird vorab geprüft) - Ladezustand beim App-Start: Spinner wird angezeigt, bis Auth-Status bekannt ist - Karten-Reihenfolge ändern: Pfeil-Buttons (▲/▼) in der Set-Detailansicht verschieben Karten und aktualisieren das `position` -Feld in der Datenbank - Set kopieren: Eigentümer können ihre eigenen Sets (öffentlich und privat) über einen „Set kopieren“-Button duplizieren - Priv-zu-Priv-Fork: Backend erlaubt nun das Forken eigener privater Sets (Endpunkt `POST /sets/{id}/fork`) - KI-Quiz URL-Fallback:

API-URL fällt auf `http://localhost:8000` zurück, wenn `VITE_API_URL` nicht gesetzt ist; 503-Antworten zeigen eine benutzerfreundliche Fehlermeldung - Gastfavoriten: Gäste können Sets als Favoriten markieren (gespeichert unter `sm_favs_guest`); beim Einloggen werden Gastfavoriten automatisch übernommen - Geführte Tour für Set-Detailansicht: startet automatisch beim ersten Öffnen eines Sets - Geführte Tour für Set-Erstellung: startet automatisch beim ersten Öffnen des Erstellungs-Dialogs - Leaderboard-Platzhalter aus Dokumentation und Code-Referenzen entfernt

Aktualisierte Dokumentationsabschnitte: Feature-Tabelle, US-003, US-014, US-041, US-050, Sidebar-Beschreibung, ChangeLog

Version 0.2.2 (2026-06-01)

Änderungen gegenüber v0.2.1: - Bugfix: Fork-Funktion kopiert Karteikarten jetzt mit korrektem `position`-Feld — Karten behalten ihre Reihenfolge im geforkten Set - Manuelle Testdokumentation in Abschnitt 14 ergänzt

Version 0.2.1 (2026-06-01)

Basierend auf `main`-Branch, Commit `cf603c0` (Lina).

Änderungen gegenüber v0.2.0: - Profilbild des Nutzers wird jetzt als Avatar in der NavBar und in der Sidebar-Fußzeile angezeigt (Fallback: Anfangsbuchstabe) - Learn- und Quiz-Buttons in der Detailansicht sind nicht mehr bei leeren Sets deaktiviert; der leere-Set-Guard in `LearnView` wurde entfernt - Bugfix: `handleAddCard` aktualisiert nicht mehr doppelt den `currentSet`-State (verhinderte Kartenduplikate) - `Edit`-Icon-Import aus `lucide-react` entfernt (nicht mehr direkt verwendet)

Version 0.2.0 (2026-06-01)

Vollständige Überarbeitung der Dokumentation, basierend auf dem Stand der `main`-Branch.

Neue Features seit v0.1.0: - Geführte Tour (`322ab03`) - Profil-Overlay und Profilbearbeitung (`3d4d234` , `184d550`) - Fork-Funktion für öffentliche Sets (`98d6cdf` , `4a41540` , `e46744c`) - Set-Titel und -Beschreibung bearbeitbar (`9282ff3`) - JSON-Import für Karteikarten (`9282ff3`) - Verbesserter Streak-Tracker mit Datums-Parsing (`28cf558`) - Light/Dark-Mode (`dcc0679`) - KI-Quiz-Migration auf google-genai SDK (`cb47bd8`) - Favoriten nur für eingeloggte Nutzer (`b332a07`) - Benutzer-Feedback bei leeren Sets beim Quiz/Lernen (`f0e33f0`)

Version 0.1.0 (2026-05-19) — Erstveröffentlichung

Grundfunktionen: - Supabase-Auth-Integration (`8cea6d6`) - Passwort-Reset-Flow (`048da9b`) - KI-Quiz-Grundstruktur (`db4dcbb`) - Flip-Card-Lernmodus (`81b7561`) - Kartensets erstellen (`fa7c4df`) - Karteikarten bearbeiten und importieren (`c2b967a`)

17. Offene Aufgaben

Priorität	Aufgabe
Hoch	Automatisierte Tests (Unit + E2E) implementieren
Hoch	Row Level Security (RLS) in Supabase konfigurieren
Mittel	Einstellungen-View vollständig implementieren
Mittel	OpenAPI-Export in <code>docs/</code> ablegen
Niedrig	Browser-Fallbacks für CSS-Features ergänzen
Niedrig	Profilbild-Speicherung zu Supabase Storage migrieren
Niedrig	Karteikarten-Bearbeitung via Backend-API (statt direkt Supabase)
Mittel	<code>sql_schema.sql</code> an Live-Datenbank synchronisieren — <code>profiles</code> -Erweiterungen, <code>friendships</code> , <code>flashcard_sets.forked_from</code> / <code>show_author</code> fehlen aktuell im Referenzschema

18. Anhänge

A) Vollständiges SQL-Referenzschema

```
-- Lernsets
CREATE TABLE IF NOT EXISTS public.study_sets (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_id    UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,
  title       TEXT NOT NULL,
  description TEXT,
  is_public   BOOLEAN NOT NULL DEFAULT FALSE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- Karteikarten
CREATE TABLE IF NOT EXISTS public.flashcards (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  set_id       UUID NOT NULL REFERENCES public.study_sets(id) ON DELETE CASCADE,
  question     TEXT NOT NULL,
  answer       TEXT NOT NULL,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

B) Pydantic-Schemas (Übersicht)

app/schemas/sets.py

```
class StudySetCreate(BaseModel):
    title: str = Field(min_length=1, max_length=200)
    description: Optional[str] = Field(default=None, max_length=1000)
    is_public: bool = False

class StudySetUpdate(BaseModel):
    title: Optional[str] = Field(default=None, min_length=1, max_length=200)
    description: Optional[str] = Field(default=None, max_length=1000)
    is_public: Optional[bool] = None
```

app/schemas/quiz.py

```
class CardInput(BaseModel):
    q: str
    a: str

class QuizGenerateRequest(BaseModel):
    cards: list[CardInput]
    count: int = 5

class QuizQuestion(BaseModel):
    question: str
    options: list[str]
    correct: int
    explanation: str

class QuizGenerateResponse(BaseModel):
    questions: list[QuizQuestion]
```

C) Nützliche curl-Befehle

```
# Health-Check
curl http://localhost:8000/health

# Öffentliche Sets abrufen
curl http://localhost:8000/sets/public

# Karten eines öffentlichen Sets
curl http://localhost:8000/sets/<set_id>/cards

# KI-Quiz generieren
curl -X POST http://localhost:8000/quiz/generate \
  -H "Content-Type: application/json" \
  -d '{"cards":[{"q":"Was ist SQL?","a":"Structured Query Language"}],"count":1}'

# Öffentliches Set forken (mit Auth-Token)
curl -X POST http://localhost:8000/sets/<set_id>/fork \
  -H "Authorization: Bearer <token>"

# Passwort-Reset anfordern
curl -X POST http://localhost:8000/auth/forgot-password \
  -H "Content-Type: application/json" \
  -d '{"username":"max_mustermann"}'
```

D) JSON-Import-Format für Karteikarten

```
[
  {
    "question": "Was ist eine Firewall?",
    "answer": "Eine Sicherheitskomponente, die den Netzwerkverkehr kontrolliert und filtert."
  },
  {
    "question": "Was bedeutet HTTPS?",
    "answer": "HyperText Transfer Protocol Secure – verschlüsselte HTTP-Kommunikation."
  }
]
```

Dokumentation zuletzt aktualisiert: 2026-06-23

Basierend auf git-Stand: [18b6bcc](#) (main branch, PR #63) + ungecommitteter Passwort-Reset-Umstellung auf OTP-Code